

Deployment von NixOS im GHG

Samson

Contents

1. Einleitung	3
2. Nix	3
2.1. Nix Language	3
2.2. Nix Package Manager und Derivationen	3
2.3. Nix Hashes	3
2.4. Nix Store	3
2.5. NixOS	3
Quellen	5

1. Einleitung

In dieser Facharbeit geht es darum, wie man Linux im GHG als Betriebssystem der Arbeit-rechner verwenden koennte. Dabei wird speziell die Verwendung der Distribution NixOS in den Vordergrund gestellte.

2. Nix

Die Grundlagen von Nix entstanden ab 2003 als Forschungsprojekt von Eelco Dolstra in seiner 2006 veroeffentlichten These [1].

2.1. Nix Language

Nix ist eine Turing-Komplette, Pure Funktionale, Domain Spezifizierte, Lazy Evaluerte und Dynamisch geschriebene Programmiersprache, in der die Beschreibung von Paketen und die Konfiguration von NixOS vorgenommen wird.

2.2. Nix Package Manager und Derivationen

Die Sprache Nix bietet Funktionen zur Beschreibung von Build-Anweisungen für Pakete. Diese werden zu Nix-Derivationen kompiliert, die (für den Nix-BUILDER) genau beschreiben, wie das Paket erstellt wird.

Den schritt des Kompilierens und des Bauens uebernimmt der Nix Package Manager. 140'000 Nix Pakete (nixpkgs) [2] werden momentan auf github:NixOS/nixpkgs gehostet.

2.3. Nix Hashes

Der Nix Hash ist wie ein Fingerabdruck einer Datei, er ist nicht nur von dem Output abhängig (z.B. einer Binary wie "vi") sondern auch von den Dependencies mit denen diese Gebaut wurde, dieser wird Benutzt um zu Verifizieren ob der Build Output exact der Spezifikation entspricht.

2.4. Nix Store

Im Nix Store, der unter `/nix/store/` liegt, werden alle Packages und Dateien unveränderlich gespeichert. Das layout sieht wie folgt aus: `Hash + Name + Version` beispielsweise fuer das Package `hello` ist der Pfad `"/nix/store/xl1h9i29pgq2q5cszjhm5wpfxfbqwyi-hello-2.12.3/bin/hello"`. Nur weil ein Package im Store ist, ist es allerdings nicht unbedingt im Pfad und kann somit nicht benutzt werden.

2.5. NixOS

NixOS ist eine deklarative Linux Distribution, man bestimmt installierte Programme und alle Konfigurationen, beispielsweise Benutzer, durch die Nix Sprache typischer Weise im Folder `/etc/nixos`. Man kann Firefox beispielsweise so installieren:

```
configuration.nix
1 programs.firefox = {
2   enable = true;
3   preferences = {
4     "privacy.resistFingerprinting" = true;
```



```
5    };  
6    };
```

Nach jeder Veraenderung der Konfiguration kann man eine neue Generation des Systems bauen und aktivieren mit `sudo nixos-rebuild switch`. Allerdings werden alte Systembeschreibungen weiterhin behalten, dass ermoeeglicht Rollbacks ohne viel Speicherplatz zu verwenden. Wenn man von Generation 1 zu Generation 2 ein Programm entfernt, bleibt dieses jedoch im Nix Store ausser man lehrt den store.

Quellen

- [1] E. Dolstra, “The Purely Functional Software Deployment Model,” 2006. Accessed: Oct. 07, 2026. [Online]. Available: <https://edolstra.github.io/pubs/phd-thesis.pdf>
- [2] NixOS, “nixpkgs: Nix Packages collection & NixOS.” Accessed: Oct. 07, 2026. [Online]. Available: <https://github.com/NixOS/nixpkgs/>